Autonomous Agents and Multi-Agent Systems (2023) 37:20
https://doi.org/10.1007/s10458-023-09603-y

Scaling multi-agent reinforcement learning to full 11 versus 11 simulated robotic football

Andries Smit[1,2] · **Herman A. Engelbrecht**[1] · **Willie Brink**[1] · **Arnu Pretorius**[2]

Accepted: 9 February 2023 / Published online: 24 March 2023
© The Author(s) 2023

Abstract
Robotic football has long been seen as a grand challenge in artificial intelligence. Despite recent success of learned policies over heuristics and handcrafted rules in general, current teams in the simulated RoboCup football leagues, where autonomous agents compete against each other, still rely on handcrafted strategies with only a few using reinforcement learning directly. This limits a learning agent's ability to find stronger high-level strategies for the full game. In this paper, we show that it is possible for agents to learn competent football strategies on a full 22 player setting using limited computation resources (one GPU and one CPU), from tabula rasa through self-play. To do this, we build a 2D football simulator with faster simulation times than the RoboCup simulator. We propose various improvements to the standard single-agent PPO training algorithm which help it scale to our multi-agent setting. These improvements include (1) using a policy and critic network with an attention mechanism that scales linearly in the number of agents, (2) sharing networks between agents which allow for faster throughput using batching, and (3) using Polyak averaged opponents, league opponents and freezing the opponent team when necessary. We show through experimental results that stable training in the full 22 player setting is possible. Agents trained in the 22 player setting learn to defeat a variety of handcrafted strategies, and also achieve a higher win rate compared to agents trained in the 4 player setting and evaluated in the full game.

Keywords RoboCup · RoboCup 2D · Multi-agent reinforcement learning · 11 versus 11 football · Soccer · Simulated robotic football

✉ Andries Smit
ap.smit@instadeep.com

Herman A. Engelbrecht
hebrecht@sun.ac.za

Willie Brink
wbrink@sun.ac.za

Arnu Pretorius
a.pretorius@instadeep.com

[1] Stellenbosch University, Stellenbosch, South Africa

[2] InstaDeep, Cape Town, South Africa

1 Introduction

RoboCup Soccer [28] is an annual event where simulated and real robots compete against each other in football. It uses football as a testing ground for algorithms which may one day help solve real-world robotic problems. In this paper, we focus on applying end-to-end multi-agent reinforcement learning (MARL) on simulated football, which would avoid a need for preconceived knowledge of football-specific strategies and lead to algorithms that could be useful in other environments. We investigate how an efficient end-to-end MARL system scales to the full 11 versus 11 2D football setting, trained entirely through self-play. We refrain from training against any handcrafted strategies and use them only for evaluation.

Top teams in recent 2D RoboCup competitions [27, 38] are relying more and more on machine learning, and specifically reinforcement learning to boost performance. These teams usually subdivide the full 2D RoboCup problem into different parts (e.g. dribbling, kicking and passing) that can be optimised individually with machine learning methods. Another line of research starts with a simplified football environment, such as the 2 versus 2 player setting, and applies end-to-end MARL on it [18, 19]. We build on that work, and investigate how it might be possible to scale such MARL systems to enable stable training in the full 11 versus 11 player setting.

As is the case in RoboCup, our aim is to have independently executing policies for the agents on the field, with each agent receiving only partial information about its environment at every time step. We try to ensure that our solution remains general without overfitting to 2D football, as we want our algorithm to be able to work on other relatively large MARL environments as well. We also limit the computational requirements for our algorithm as much as possible so that researchers with only one GPU can still use it. The main contributions of this paper are as follows:

- providing evidence to help understand why RL algorithms have not been successful in the full 11 versus 11 player setting to date, with reference to well-known challenges of correlated experience, non-stationary environment dynamics and self-play strategy collapse [24];
- proposing a new variation of the PPO algorithm [29] to address the above mentioned stability issues and exhibit stable training in the full 11 versus 11 player setting, with only one GPU;
- experimentally verifying that the proposed algorithm can learn good strategies that defeat a variety of handcrafted strategies in just over two days of training;
- experimentally verifying that agents trained in the full 22 player football game achieve a higher win rate than agents trained in the 4 player setting and evaluated in the full game, thereby validating the need to train in the full 22 player setting.

In Sect. 2, we highlight existing work on applying learning algorithms to derive strategies in football environments. In Sect. 3, we present our lightweight simulation environment for training and evaluation. In Sect. 4, we hypothesise what the challenges are in applying MARL in the full 22 player setting, and propose possible solutions. By applying these solutions we show in Sect. 5 that agents trained in the full 22 player environment can learn to outperform a variety of handcrafted strategies, as well as agents trained in the 4 player setting and evaluated in the full 22 player game. We also perform an ablation study to demonstrate how some of the main improvements proposed in this work contribute to the final result. In Sect. 6, we conclude with a discussion and thoughts on how to expand this work in the future.

2 Related work

An agent playing football faces many distinct (but not mutually exclusive) tasks, like dribbling, kicking the ball, recovering from a fall, etc. One way of approaching this large problem-set is to compartmentalise the tasks into hierarchies. An agent can then learn lower-level behaviours like running and getting up before learning higher-level strategies such as passing the ball. One of these hierarchical machine learning paradigms is called layered learning [31] and has been successfully applied in RoboCup's 3D simulated league, even leading to 2014's winning team [21]. A problem with layered learning is that when lower-level modules are learned and frozen, agents might not be able to learn optimal behaviours overall. Alternatively, learning all lower-level modules simultaneously might be too slow or even unstable. MacAlpine and Stone [21] addressed this problem with overlapping layered learning, where lower-level modules are partially updated during learning, for increased final performance. They successfully learned 19 layers of behaviours and optimised over 500 parameters. Up to two agents were trained in a football environment, and learned behaviours were evaluated in the full 22 player game. In our research we opt to use a simpler 2D environment with less complicated environment dynamics. We do not use layered learning and instead derive everything, from individual agent skills to high-level team strategies, directly from competitive self-play. We also train our team in the full 22 player setting, which we believe will allow for better team strategy formation.

The winner of the 2019 RoboCup 2D simulation league was a team called Fractals2019 [27]. Their main strategy was to combine an evolutionary algorithm with guided self-organisation. They combined artificial evolution with human innovation, a method also called human based evolutionary computation (HBEC). As mentioned in their work, Prokopenko and Wang [27] rely on methods such as particle swarm based self-localisation, tactical interaction networks, dynamic tactics with Voronoi diagrams, bio-inspired flocking behaviour, and diversified opponent modelling. HBEC is used to determine what combination of these and other techniques provides the best results by manually (or automatically) adjusting hyper-parameters and measuring an agent's fitness, e.g. using goal difference with respect to other agents.

HELIOS [1, 2], which won the RoboCup 2D simulation league in 2010, 2012, 2017 and 2018, uses a team formation model that implements Delaunay triangulation [16], to generate points that are sufficiently spread out from one another. This allows the agents in the HELIOS team to cover greater areas of the field, without bunching. The algorithm also employs a tree search method to search through sequential ball kicking actions among multiple players on the same team, in order to determine which actions the agents should take. A goal score criterion is used as an evaluation to weigh different action sequences.

Searching through all possible combinations of ball passes quickly becomes intractable. To help the search algorithm, Akiyama et al. [1] prune all actions that are not part of the intended tactics that they want their team to exhibit. To do this, they use a support vector machine (SVM) classifier to determine whether a given action is part of the intended tactic or not. To generate the label set for this SVM, they extract game data from different training runs where they manually label actions, using a graphical user interface, as being part of the intended tactic or not.

More recently, a form of layered learning has been applied on a 3D simulated humanoid body with 56 degrees of freedom [19]. Some of the complexity of the football environment was reduced, by making the ball bounce back from the sidelines and removing penalties altogether. Agents were trained in three stages. Firstly, agents would perform imitation learning

for low-level movement skills, using motion capture data from real human players. In the second phase, individual agents would learn mid-level skills by performing basic drills, e.g. running and dribbling. In the third phase, MARL was used with population-based training (PBT) in a 2 versus 2 player setting, with 16 independent agents that trained in parallel on multiple GPUs. The agents learned to play competent football in this setting and the authors also presented evidence that the agents could pass the ball and recover from falls. A multi-headed attention mechanism [33] was used, which we also use in our work. Unfortunately, the computational requirements for agent policies scale quadratically with the number of agents, making it expensive to compute agent actions in the full 22 player setting. In our work, we adapt the network to scale linearly with the number of players on the field, which enables significantly faster training in the 22 player setting.

Liu et al. [18] also focused on the 2 versus 2 player simulated setting. However, their environment is much simpler than the one used by Liu et al. [19]. Their environment is similar to the 2D RoboCup environment but has fewer players, no penalties and fewer delays (e.g. for throw-ins) in the game. They were able to train competent agents using end-to-end RL, with PBT to perform automatic reward shaping and hyperparameter tuning. 32 independent agents were trained in parallel on multiple GPUs. Focusing on 2 versus 2 player football reduced the computational resources required to achieve competent results, and agents were not evaluated in the full 11 versus 11 player game. We opt to not use PBT and will rely on only one GPU for training in the full 11 versus 11 setting. This should make it easier for other researchers, who might have constrained computational resources, to build upon this work.

In competitive games it is important to vary the opponents that a policy is trained against. If a policy trains against only one opponent its strategy might become exploitable. For efficient learning one also needs to balance the strength of these opponents so that the learning policy maintains a balanced win/loss rate throughout training. Brown [6] proposed the fictitious self-play algorithm, which is a game theoretic concept where agents are guaranteed to converge to a Nash equilibrium strategy in 2 player zero-sum games. A Nash equilibrium strategy is an optimal solution where no agent has any incentive to change its strategy given the other strategies. Within fictitious self-play the current learning policy is periodically added to a league of opponents. To combat exploitability, fictitious play requires that each policy should find the best response strategy to the average over the policy's history of strategies. However, simply taking the average over all the opponent's previous strategies might lead to slow convergence. This is because the initial strategies of policies might be relatively weak and it might be better to place more emphasis on strategies encountered later in training. Leslie and Collins [17] proposed an update to fictitious play, called generalised weakened fictitious play, which retains the Nash equilibrium convergence guarantee of fictitious play but allows for more flexibility in update rules. Their algorithm allows for faster convergence by sampling stronger opponents more frequently than weaker ones. Heinrich et al. [11] extended the work of Leslie and Collins [17] by proposing the first self-play hybrid algorithm, that combines supervised learning and reinforcement learning. It is an open question whether their algorithm inherits the guarantee of converging to a Nash equilibrium.

In 2019, Berner et al. [5] created an artificial team that learned through reinforcement learning and self-play to defeat the reigning Dota 2 human world champions. This research is of interest to us as our football environment also consists of multiple agents and has a self-play component to it. Furthermore we also use proximal policy optimisation [29] which is a popular reinforcement learning algorithm. While Berner et al. [5] opted to train one controller to control all five players simultaneously, we opt to have independently executing policies to better align with the rules of the 2D RoboCup league. The most useful aspect that we take from Berner et al. [5] is their league training, where the latest team plays not

only against itself, but also against previous versions of itself. AlphaStar [22] employs a similar strategy by placing a league of independently learning agents against each other. This prevents a common pitfall when using self-play, called strategy collapse, where a team only optimizes against its current strategy. In doing so, its strategy can become brittle and exploitable as it is not trained to be robust against a variety of other opponents.

Recent work by Yu et al. [37] showed that the proximal policy optimisation (PPO) algorithm can work well in the cooperative multi-agent setting, with slight adaptations such as improving inputs to the value function using agent specific global states, improving training data usage, and setting the PPO clipping value lower. We also use these recommendations in our work, and add a few additional improvements as described in Sect. 4.2.

There have been many recent works on addressing the sparse reward problem in multi-agent reinforcement learning through various forms of reward shaping. Potential based rewards [8, 36] introduce domain specific knowledge to the reward function to reduce exploration times of agents. This is done by providing additional rewards to actions that are recommended by domain experts. Similarly, difference rewards [9] favour individual agent actions that are aligned with the overall objective of the multi-agent system. In this work we try to provide as little domain specific knowledge to our agents as possible. We only include an initial shaped reward, which we remove after the agents learn to score goals.

3 Simulation environments

We now describe the two environments used in this work.

3.1 Custom football environment

A future goal of this work is to create a team of 11 agents that can compete in the 2D simulated RoboCup league. To this end, we adapted the 2D RoboCup simulator to work with a Python interface and trained simple agents using reinforcement learning in this environment. The complete Python wrapper[1] is available inside the multi-agent reinforcement learning framework called Mava [26], further described in Sect. 5.1.1.

To simplify the football problem further, we constructed a simpler 2D football environment[2] for experimentation with a runtime more than 25× faster than the fastest we could get from the 2D RoboCup simulator (as shown in Sect. 5.2). We created our own environment to further improve on simulation times and to enable only partial observations of the field to players, similar to the RoboCup simulator. Our environment is able to run anything from 1 versus 1 player matches to the full 11 versus 11 setting. Fast step times are achieved through parallel matrix operations on limited game physics. Players can only interact with the ball and not hit each other, which reduces the number of calculations necessary per step. The ball bounces from the sidelines, eliminating the need for throw-ins. Generated views of the 2D environment are provided in Fig. 1.

Our custom environment is similar to the one of Liu et al. [18]. We chose to reproduce their environment's agent embodiment as it seemed closer to how real humanoid robots kick with their limbs. RoboCup's 2D embodiment could also be a possible alternative for future work. In this custom environment an agent can dribble the ball by repeatedly hitting it with the central circle part of its body, or kick the ball using its side legs. The short yellow line on a player indicates the direction it is facing. Once the ball has crossed the goal line the

[1] Link: https://github.com/instadeepai/Mava.

[2] Link: https://github.com/DriesSmit/MARL2DFootball. We include all baseline bots as part of the release.

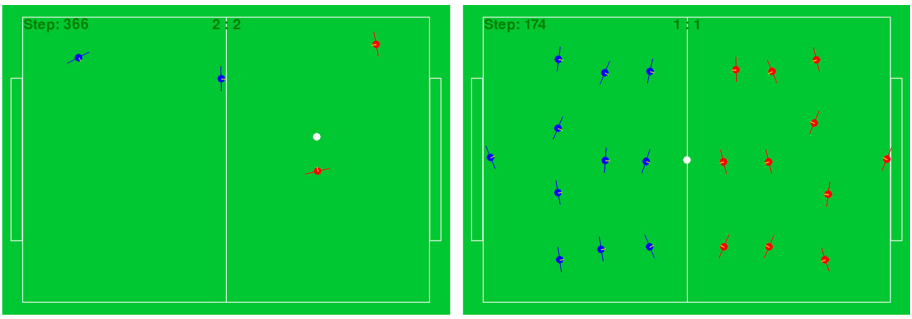


Fig. 1 Two top-down graphical views of the 2D football environment. Left: 4 player game. Right: full 22 player game. The blue players' goalpost is on the left side, and the red players' goalpost is on the right. The goalposts are made larger to prevent agents from completely blocking off their own goals (Color figure online)

score of the game is updated, which is displayed at the top centre of the screen. A game consists of a predefined number of steps in which players can execute moves. The current environment step is displayed at the top left of the screen. After a goal is scored the players re-spawn to their starting positions and orientations, with some added noise if needed. The ball can also be spawned at a random location if needed. After the game is completed, the score determines the winner or whether it is a draw.

The players act in the environment by providing a continuous valued action vector of size 2. The first entry signifies movement along the axis a player is facing, with forwards being positive and backwards negative. The second entry allows the agent to rotate, where a negative number represents anti-clockwise rotation, 0 no rotation and positive clockwise rotation. These actions update the position and rotation of each player over short distances. Stamina and fatigue, which are included in the RoboCup simulator, are omitted in this simplified environment. The agents receive a visual input and an indication of how many steps remain before the game is completed. To keep the problem close to RoboCup's setup, each agent is limited to a partial observation of the field through a 180° vision cone. This means that the agent can see only what is in front of it. We decided that the vision system should be egocentric (from the agent's perspective), even though this generally leads to a harder problem compared to a fixed top-down vision system. The egocentric view is closely related to how the 2D RoboCup simulated league is set up and also seems more realistic. The agents are provided with the egocentric coordinates and velocities of all objects (ball and other players) in their 180° vision cones. This is similar to the environment of Liu et al. [18], although in their case complete state information is provided to the agents. Each agent also receives the absolute coordinates of its location on the field, as well as the last actions of all the agents it can see. Lastly, each agent also receives its starting field position. This may encourage agents to learn differentiated behaviours depending on what position they are playing.

To facilitate initial learning, a temporary reward shaping signal is provided to the agents. This reward is necessary because initially the likelihood of scoring a goal is quite low. The formula for this reward shaping is

$$r_{i,t} = g_{i,t} + 0.1b_{i,t} + 0.05a_{i,t}, \tag{1}$$

where $r_{i,t}$ represents the reward received by player i at a given time step t. Here $g_{i,t}$ represents 0 if no goal has been scored in step $t-1$, $+1$ if the player's team scored and -1 if they were scored against. The variable $b_{i,t}$ represents the magnitude of the ball's velocity towards the opponents' goal and $a_{i,t}$ the magnitude of the player's velocity towards the

ball's coordinates. This incentivises the players initially to move towards the ball and kick it towards the opponents' goal. Once the players score at least one goal in more than 75% of the games, the reward shaping is removed. From that point on the players rely only on the $g_{i,t}$ term for training. We chose weightings of 0.1 and 0.05, as we would like the ball moving towards the goal to provide more reward than the ball moving away from the agent. Other than this constraint, the values are quite arbitrary and we found that the agents learn with other weightings too.

After reward shaping, the agents receive zero reward until a goal is scored. The team that scored the goal receives a $+1$ reward and the other team receives a -1 reward. A Python code snippet for a simple RL training routine in our simulation environment is presented below. This routine shows how the environment can be used to train a team of agents using RL.

```
# Initialise the football environment.
football_env = FootballEnvWrapper(num_per_team=11)

# Initialise the teams.
rl_team = RLTeam()
random_team = RandomTeam()
teams = [rl_team, random_team]
while True:
    # Reset the agents' internal states.
    for t_i in range(len(teams)):
        teams[t_i].reset_brain()

    # Start a new game.
    observations, states, rewards = football_env.reset_game
    ()

    # Store the initial observations, states and rewards
    received
    # from the environment.
    rl_team.observe_first(observations[0], states[0],
    rewards[0])

    done = False
    while not done:
        # Get the agents' actions.
        actions = []
        for t_i in range(len(teams)):
            actions.append(teams[t_i].get_team_actions(
    observations[t_i]))

        # Step the environment.
        observations, states, rewards, done = football_env.
    step(actions)

        # Display the football screen.
        football_env.display_screen()

        # Store the observations, states and rewards for
    this
        # environment step.
        rl_team.observe(observations[0], states[0], actions
    [0], rewards[0],
                                done)

    # Perform an agent update step.
    rl_team.trainer_step()
```

The code for this RL team can be placed inside the `RLTeam` class. Note that the team collects experience through the `observe_first()` and `observe()` functions and then updates its internal networks when `trainer_step()` is called. One can experiment with different algorithms for the `RLTeam` class, that can range from independent policies per agent (as is done in this work) to one policy controlling the entire team.

3.2 2D RoboCup simulation environment

Our main focus is on training and evaluating in our custom environment with its simpler dynamics and faster runtime. However, we would also like to see whether it is possible to apply our learning algorithm to the 2D RoboCup simulation environment. In this environment players receive partial egocentric observations of all the objects they can see in their 90° field of view (half the visibility provided in our custom environment). Furthermore, the RoboCup server does not provide players with their absolute coordinates as observations, but rather relative coordinates of the landmarks on the field visible to each player. To simplify learning we opt to not provide raw landmark coordinates to each player, and instead we calculate the translation and rotation necessary to map these relative coordinates to absolute coordinates. The calculated translation and rotation are then used to estimate the player's absolute coordinates (with added noise provided by the simulator), which we provide to the player.

In our wrapper we scaled all the actions to be between -1 and 1, as that is the range of our algorithm's output. In the RoboCup environment the action space is more complicated than in our custom environment. Players are allowed to perform one of five main actions per step, namely dash, kick, tackle, turn or not moving. Dash allows a player to move around in any direction relative to its body, where moving in the forward direction consumes only half the stamina than moving backwards. Each player has a stamina bar that increases by a set number of points each step. When a player executes any action, except the no move action, stamina gets used. When a player's stamina reaches zero, the effectiveness of actions (accuracy and power) is drastically reduced. The player can then choose to either wait for more stamina or move with this reduced capability. Furthermore, a player can move faster in the forward direction than sideways or backwards.

The kick action allows a player to kick the ball in some chosen direction relative to its body, if the ball is within the body radius. The player's kicking power reduces as the kicking angle moves from the forward direction (0°) to the backward direction ($\pm 180°$). The tackle action is used to kick the ball in a direction chosen by the player and can be used if the ball is just outside the player's body radius, but more noise is added to the direction of the ball compared to normal kicking. The tackle action can also be used to foul another player close by, which might result in a penalty if the referee observes it (with some probability). The turn action allows the player to turn in some chosen direction relative to its body.

Only one of the above five actions can be performed at a time, which is different to our custom environment. Players can simultaneously perform additional actions such as turning their heads to look in a different direction than their body is facing, and changing their viewing quality to receive visual information more often (e.g. every step) but in a narrower field of view, or less often (e.g. every second step) but in a wider field of view. To simplify training we fix the head to face in the direction of a player's body, and we also fix the viewing angles. We replace the goalkeeper (who can catch the ball) with another normal player to keep the action space consistent for all players.

Table 1 The continuous action space used by the wrapped RoboCup environment

Action index	Action description
0	Selection score for the dash action
1	Selection score for the kick action
2	Selection score for the turn action
3	Selection score for the tackle action
4	Selection score for the no move action
5	Dash power (-1 backwards to $+1$ forwards)
6	Dash direction ($-90°$ to $90°$)
7	Kick power (0 to 1)
8	Kick direction ($-180°$ to $180°$)
9	Turn direction ($-180°$ to $180°$)
10	Tackle foul (0 for not foul and 1 for foul)
11	Tackle direction ($-90°$ to $90°$)

All actions are scaled and normalised to have continuous values between -1 and 1

RoboCup uses a hybrid action space where both discrete and continuous actions can be selected. Each of the main actions has continuous controls associated with it that determine in which direction and, when applicable, with what power to execute that action. However, our learning algorithm works only with continuous actions and we therefore need to convert this hybrid action space to a continuous one. To do so, we add five additional continuous actions to determine which of the main actions to take. The highest of these five values will indicate which action to take, along with its corresponding continuous control outputs. The complete action output vector is shown in Table 1.

4 Scaling to the 11 versus 11 player setting

We now investigate what is needed to scale end-to-end reinforcement learning systems to work in the full game of football. We first present some of the challenges with scaling RL algorithms in the football setting, and then propose possible solutions to these issues.

4.1 Challenges

The aim of this work is to get an RL system to learn, from scratch, how to play competitive football in a full game of 11 versus 11 players. Football is already a hard problem when training only, say, 4 agents. However, it becomes significantly harder for standard RL algorithms to achieve good results when training in the full 11 versus 11 game. Reasons for this are presented next.

Non-stationary environment: Training in environments where many agents are learning simultaneously is known to be a hard problem in RL [7]. In multi-agent reinforcement learning, from the perspective of each agent, the environment itself changes over time and this can destabilise training. Assigning value to the action an agent takes at a given step can be hard, as other agents also perform actions at the same time.

Partial observability: Agents can only see in front of them and therefore might not know where all the objects in the environment are. This makes training harder as an agent

additionally needs to learn to use its memory to remember important information which might not be observable at every time step [23].

Sparse rewards and credit assignment: Environments with sparse rewards also complicate learning [30]. In football, agents typically receive only a few non-zero rewards in an entire game. This makes it hard for agents to determine which (if any) of their previous actions were responsible for these rewards.

Self-play: Agents playing against copies of themselves can be an effective way to introduce curriculum learning into the training process [3]. The agents might start out against weaker opponents and as they get better, the opponents also get better. Therefore the agents gradually learn better strategies over time. Using naive self-play where one network plays only against itself can lead to strategy collapse [24]. This generally occurs when a policy only focuses on defeating the latest strategy and ignores all previous strategies found over the course of training [15]. Furthermore, if a network update leads to a worse strategy it might be hard to detect, as both teams use this new strategy. Therefore the reward might not change to indicate some deterioration in strategy.

Computational overhead: When training with a large number of agents it takes significantly longer to process an episode, because at each step, 22 neural network forward passes need to be computed. Every agent's interaction with the environment also needs to be calculated, along with updated state and observational data. It may be argued that there are more agents generating experience per episode, which could counteract episodes taking more time. However, experiences generated in the same episode are much more correlated than across different episodes, which can lead to catastrophic forgetting [14] and deterioration of learning. Furthermore, each individual player's experience contains a much smaller proportion of ball interactions. Therefore it is harder for them to learn basic ball handling skills.

4.2 Possible solutions

For this work, we focus on using multi-agent proximal policy optimisation (MA-PPO) [34], where policy updates can be controlled more easily. This reduces the likelihood of policy collapse from the non-stationary nature of multi-agent environments. MA-PPO extends PPO [29] to the multi-agent setting by using a centralised critic when training, and was shown to work well in cooperative settings [37]. We wish to extend MA-PPO for the mixed cooperative-competitive setting. In order to address the challenges listed in Sect. 4.1 we propose Algorithm 1, where each iteration adds one episode's experience to the data buffer and performs updates on the networks based on the collected experiences. An episode is played out between the learning team and the Polyak averaged opponent. The experience of the learning team is sent to the trainer and is used to update the learning team's networks. The trainer uses the standard PPO training setup [29] for each agent, with a clipped policy loss function and entropy term. The critic is used to derive an advantage function which determines how to update the policy. The opponent's networks are updated by slowly adjusting them towards the learner's networks. In Algorithm 1, discounted_return($\boldsymbol{r}_1$) calculates the sum of discounted future rewards at every time step, and average_score($\boldsymbol{r}_1$) calculates the average game outcome (win: 1.0, draw: 0.5, lose: 0.0) over all episodes in $\boldsymbol{r}_1$. Lastly, entropy($\pi_{\theta_1}(\boldsymbol{a} \mid \boldsymbol{o}_1)$) calculates an entropy value for the action ($\boldsymbol{a}$) distributions of the policy given the observations $\boldsymbol{o}_1$. The various improvements over the standard PPO implementation in the MARL setting are presented next.

Clipping value: As recommended in previous work [37] we set a small clipping value for the policy loss function to avoid training collapse. The clipping value is used to control

Algorithm 1 Our multi-agent PPO setup.

Require: $\theta_1, \phi, \lambda_\pi, \lambda_v, \alpha_v = 0.5, \alpha_p = 1.0, \alpha_e = 0.01, \alpha_s = 0.01, \epsilon_p = 0.1, \epsilon_s = 0.55, epochs_per_step = 5$

▷ We use a smaller ϵ_p clipping value and a smaller *epochs_per_step* value than in the standard PPO implementation, for increased training stability, and a small α_s value to slowly update the opponent strategy. We also use a different set of weights for the opponent agents (θ_2) than for the learning agents (θ_1).

$\theta_2 \leftarrow \theta_1$
$\mathcal{D} \leftarrow \emptyset$
for each iteration **do**
  for t **in** environment steps **do**
    $\boldsymbol{s}_t \sim p(\boldsymbol{s}_t \mid \boldsymbol{a}_{1,t-1}, \boldsymbol{a}_{2,t-1}, \boldsymbol{s}_{t-1})$
    $\boldsymbol{s}_{1,t}, \boldsymbol{s}_{2,t} \leftarrow \boldsymbol{s}_t$ ▷ Each team has its own state value.
    $\boldsymbol{o}_{i,t} \sim p(\boldsymbol{o}_{i,t} \mid \boldsymbol{s}_t)$ **for** $i \in \{1, 2\}$
    $\boldsymbol{a}_{i,t} \sim \pi_{\theta_i}(\boldsymbol{a}_{i,t} \mid \boldsymbol{o}_{i,t})$ **for** $i \in \{1, 2\}$ ▷ Perform two batched forward passes (one per team) for faster inference.
    $\boldsymbol{r}_{1,t} \sim p(\boldsymbol{r}_{1,t} \mid \boldsymbol{s}_t, \boldsymbol{a}_{1,t}, \boldsymbol{a}_{2,t})$
    $\mathcal{D} \leftarrow \mathcal{D} \cup (\boldsymbol{o}_{1,t}, \boldsymbol{s}_{1,t}, \boldsymbol{a}_{1,t}, \boldsymbol{r}_{1,t})$
  end for
  for each trainer step **do**
    $\boldsymbol{o}_1, \boldsymbol{s}_1, \boldsymbol{a}_1, \boldsymbol{r}_1 \sim \mathcal{D}$ ▷ Sample experience from the learning team.
    for epoch **in** *epochs_per_step* **do**
      $V_1 \leftarrow V_\phi(\boldsymbol{s}_1)$ ▷ Value function uses full state information.
      $V_{target} \leftarrow \text{discounted_return}(\boldsymbol{r}_1)$
      $A_1 \leftarrow V_{target} - V_1$
      $V_{loss} \leftarrow A_1^2$
      $u_1 \leftarrow \frac{\pi_{\theta_1}(\boldsymbol{a}_1|\boldsymbol{o}_1)}{\pi_{\theta_1 \text{old}}(\boldsymbol{a}_1|\boldsymbol{o}_1)}$
      $p_{loss} \leftarrow -\min(u_1 A_1, \text{clip}(u_1, 1-\epsilon_p, 1+\epsilon_p)A_1)$
      $J(\theta_1, \phi) \leftarrow \mathbb{E}\left[\alpha_v V_{loss} + \alpha_p p_{loss} - \alpha_e \text{entropy}(\pi_{\theta_1}(\boldsymbol{a} \mid \boldsymbol{o}_1))\right]$
      $\theta_1 \leftarrow \theta_1 - \lambda_\pi \nabla_{\theta_1} J(\theta_1, \phi)$
      $\phi \leftarrow \phi - \lambda_v \nabla_\phi J(\theta_1, \phi)$
    end for
    if average_score($\boldsymbol{r}_1$) $> \epsilon_s$ **then** ▷ When necessary, freeze the opponents' weights to stabilise training.
      $\theta_2 \leftarrow \alpha_s \theta_1 + (1 - \alpha_s)\theta_2$ ▷ Slowly update the opponents' weights.
    end if
  end for
end for

how far a policy can deviate from the original policy that was used when the experience was collected. If the policies deviate too much the experience might become unreliable as the expected returns (discounted sum of future rewards) were estimated for the old policy. This is especially important in the multi-agent setting where 22 actions are taken per step instead of just one. Furthermore, by making only small updates to the team policies we try to keep the state distribution close to what the critic has experienced in the past. The critic therefore provides better estimates of the discounted return, which reduces noise in the learning process. We use a clipping value of 0.1 instead of the default value of 0.2 for PPO.

Training data usage: A powerful implementation in reinforcement learning is mini-batching, where a batch of experience is split into smaller batches and then trained on multiple times, allowing experience to be used more than once. This helps increase the sample efficiency of our algorithm. However, as noted by Yu et al. [37], too many updates on the same experience can lead to degraded performance in the multi-agent setting. We therefore use a reduced epoch size of 5, instead of 64 as in the original PPO implementation [29]. We found experimentally that this provides the best benefit ratio between experience reuse and stability in training.

Environment-provided global state: The last method we borrow from Yu et al. [37], originally presented in Lowe et al. [20] and Foerster et al. [10], is using the global state as input for each agent's critic. We use the popular decentralised execution with centralised training architecture, allowing full state information to be fed into the critics and only partially observed inputs into the policies. The critics can then be discarded at test time. Using full state information allows the critics to better model the value function because no information is hidden from them.

Parallel computation: The Mava framework [26], which is based on Acme [12], uses Launchpad [35] to asynchronously train and run separate processes for the experience generators (workers) and trainer. This allows for better use of the computational resources available as both the CPU and GPU can be utilised to their fullest. For our experiments, we use 10 worker processes (CPU cores) and one trainer (GPU) running on the same machine. Further specifications on the computational hardware used are provided in Sect. 5.1.3.

Polyak averaging: While the above methods help stabilise and speed up training they do not directly address the problem of strategy collapse inherent in self-play. The learning team might overfit to defeating a current strategy, instead of finding a more general (balanced) strategy that works against more than one team. To avoid this, the team of 11 agents plays against a slow-moving (Polyak averaged) version of the latest network. The Polyak averaged opponent approximates an average over recent strategies discovered by the training team. We use the following equation to update the slow-moving network:

$$\theta_2 \leftarrow \alpha_s \theta_1 + (1 - \alpha_s)\theta_2, \tag{2}$$

where θ_2 contains the slow-moving average network weights (opponents) and θ_1 the current network weights that are being updated using the PPO algorithm. The α_s value is a constant that we set to 0.01. We apply this update at the end of every PPO trainer step.

League training: As an alternative to Polyak averaging we also experiment with directly training against a league of opponents. In our work we save the policy parameters every 10 iterations to form part of this league. We do not use the same sample update rule as Berner et al. [5]. Instead we decide to design our own simpler update rule. We want the sampling rule to sample stronger opponents with higher probabilities, and weaker ones with lower probabilities. We also want the probability of sampling a very weak opponent to approach 0 as the win rate of that opponent approaches 0. With this in mind we define the probability of sampling an opponent team to be

$$p_i = \frac{(w_i)^\psi}{\sum_{j=1}^{N} (w_j)^\psi}, \tag{3}$$

where w_i is the win rate of the opponent over the current policy. If $w_i = 1$ the opponent always defeats the current policy, and if $w_i = 0$ the policy always defeats the opponent. ψ is a user defined non-negative real value. If $\psi = 0$ all previous opponents will be sampled at equal probability, similar to the classic fictitious play algorithm [4]. If ψ is set to be a large value only the opponent with the highest win rate will be sampled. This is approximately the same as learning against only the current best policy. We set $\psi = 2$ which prioritises opponents with higher win rates, but still samples opponents that have slightly lower win rates. Opponents with win rates close to zero are very rarely sampled. To calculate the win rate for each opponent we average the past 1000 games played between the policy and the opponent. If fewer than 100 games have been played we keep the win rate at 0.5. This prevents a new opponent from never being sampled again if it loses an initial game.

Opponent freezing: To further combat strategy collapse the weights of the slow-moving average opponent team are frozen if the learning team has an average game outcome score of less than 0.55. Thus the slowing-moving average strategy waits for the learner if the learner is struggling to beat it consistently. This stabilises training by preventing the Polyak averaged opponent from updating too fast.

Shared weights: We also use shared policy and critic networks for each agent in a team [18]. This means that all the generated experiences can be used to train one policy and one critic network, which drastically reduces the number of parameters to be tuned compared to using an individual network for each agent. A player's starting position can still be used to learn different strategies depending on its position in the team. This starting position is provided as input at every step throughout an episode.

Policy batching: To decrease the time it takes to generate actions in the environment, we process the policy forward pass of each agent in a team in parallel through batching. Therefore each team only performs one batched forward pass (11 agents per batch) instead of 11 individual forward passes. According to our experiments, this simple change can make execution up to 5 times faster.

Network setup: The critic network setup used in this work is shown in Fig. 2. It takes in general information such as the ball's location and time, and encodes it using a 2-layer feed-forward neural network called `encode_ff`. The encoded information is fed to `query_ff` to generate query embeddings for the multi-headed attention layer. The state information of each of the team's players is fed through `encode_team_ff` to generate team embeddings. The state information of each of the opponents is fed through `encode_opp_ff` to generate opponent embeddings. These player embeddings are all concatenated with the query embeddings (concatenation blocks are not shown to simplify the diagram) and passed through `value_ff` to generate values for the attention mechanism. The values are concatenated and, with the query embeddings, fed to the multi-headed attention layer [33]. The attention layer ensures order invariance by using a weighted softmax operation to determine how much of each of the inputs (values) should be represented in the output values. Finally, the weighted output values of the attention mechanism as well as the ball and time encoded embeddings are sent through `out_ff` to output a scalar value representing an estimate of the expected discounted return for the team. The expected discounted return is the expected discounted sum of future rewards the agents will still encounter in the current episode. This predicted value is subtracted from the real returns encountered to generate an advantage estimate [25], which indicates whether the actions agents took were better or worse than expected, and can then be used to make these actions more or less likely in the future.

Similar to Liu et al. [19], we use an attention-based component for the policy and critic networks. It allows for input vectors of different sizes to be fed to the network, depending on how many agents are observed in the case of the policy, or how many are on the field for the critic. Thus agents can, for example, be trained on 2 versus 2 games and still play 11 versus 11 games. Liu et al. [19] fed a pairwise concatenation of agent embeddings to their value network. We choose not to use pairwise embeddings as they scale quadratically in computational cost as the number of agents increase, and instead opt for a simpler embedding per agent approach. Our system scales linearly with the number of agents, which enables training in the full 22 player setting.

The policy network also uses the configuration in Fig. 2, with minor alterations. Due to the environment being partially observable we use a recurrent policy with two layers of LSTM units after the `out_ff` layer. This gives the agents the capability to remember previous observations and internal calculations. We also feed the player's location, with the ball and time observations into `encode_ff`. Lastly, we only feed in teammates and opponents that

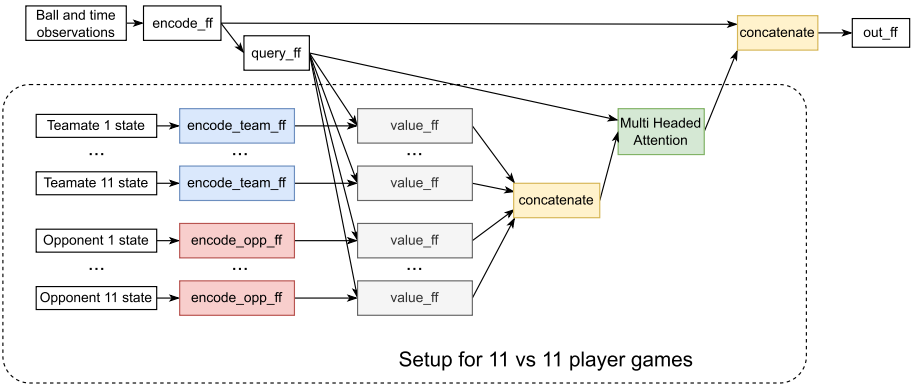


Fig. 2 The network setup for the critic network used in our PPO implementation. This network is designed to be agent order invariant and therefore produces the same output for any order in which agents are presented as input. It can also take in a varied number of agents due to the use of a multi-headed attention layer. The `encode_ff`, `query_ff`, `encode_team_ff`, `encode_opp_ff`, `value_ff` and `out_ff` blocks all represent 2-layer feedforward neural networks

the player can see in its 180° vision range, into the attention mechanism. All observations of the ball, teammates and opponents are also converted to the egocentric viewpoint.

5 Experiments

We now evaluate how the PPO algorithm, with the various improvements proposed in the previous section, performs in our 22 player football environment. We compare our team trained in the 22 player setting with different handcrafted strategies as well as our algorithm trained in the 4 player setting. We provide an ablation study which indicates how some of the main improvements influence the team's performance. We then investigate possible reasons why the team trained in the 22 player setting might be performing better than a team trained in the 4 player setting. We end off this section by showing how our algorithm can be transferred to the 2D RoboCup environment. Details on the hyperparameters we use can be found in Appendix 1.

5.1 Experimental design

Next we discuss some of the experimental design decisions made for this work.

5.1.1 MARL framework

In this work, we leverage the open source multi-agent reinforcement learning framework called Mava [26]. Mava allows for splitting of the training procedure into experience generators (workers) and a network updating process (trainer), which can be run in parallel to increase efficiency.

5.1.2 Handcrafted opponents

The aim of this research is to show that stable training is possible in the full 11 versus 11 football game. To evaluate that our PPO agents do get better over time (using only self-play) we evaluate them against three opponent teams with fixed, handcrafted strategies. We have a team of agents that move randomly on the field, representing our control team, as well as two other teams where the more difficult one is loosely inspired by real football strategies. The three teams can be described as follows:

- Random (easy): This team simply outputs random actions and is used as a control.
- Naive (medium): Players search for the ball and move towards the ball if it is found. If a player is close enough to the ball, it kicks it in the direction of the opponent's goal. Therefore, this team disregards cooperation and simply tries to score goals as fast as possible.
- Teamwork (difficult): This team consists of defensive and offensive players. The offensive players try to move towards the ball and kick it towards the opponent's goal (similar to the naive team). The defensive players form a protective semi-circle around their own goal, moving only when the ball is close to their goal and then trying to kick the ball towards the side of the field. This strategy performs better than naively chasing the ball.

5.1.3 Hardware specifications

A key motivation for this work is also to reduce the computational requirements as much as possible, to allow researchers with limited resources to reproduce and build upon our results. For every training run a single machine with 68 GB of RAM was used. We have 10 workers running on a single CPU with 12 cores (Intel® Core™ i7-8700K CPU @ 3.70GHz). One trainer runs on a GP104 (GeForce GTX 1080) with 8 GB of RAM.

5.2 Experimental results

We next present the main findings of this work.

5.2.1 Environment performance

In this section we investigate the relative speed differences of our custom football environment,[3] DeepMind's simplified football environment[4] [18] and our wrapped version of the RoboCup 2D simulation environment.[5] The execution speeds achieved in each of these environments are shown in Fig. 3, where each player is controlled by a random policy with negligible internal compute. Here we focus only on the environment step speed.

As can be seen, the custom environment has the highest steps per second throughput. In the 1 player per team setting, the custom environment runs at 4670 steps per second, which is 7.6× faster than the wrapped RoboCup environment and 13.6× faster than the DeepMind environment. In the 11 players per team setting, the custom environment is 25× and 102× faster than the RoboCup and DeepMind environments, respectively. With 22 players on the field our custom environment runs at 1533 steps per second. This does not imply that

[3] Our custom environment: https://github.com/DriesSmit/MARL2DSoccer.

[4] DeepMind's environment: https://github.com/deepmind/dm_control.

[5] Wrapped RoboCup 2D environment: https://github.com/instadeepai/Mava.

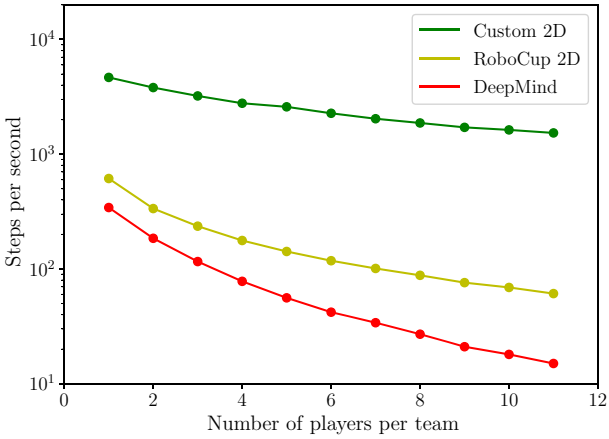


Fig. 3 The environment steps per second throughput for different 2D football environments. The horizontal axis represents the number of players per team, and the vertical axis the steps per second of the environment in log scale

the custom environment is better than the other two environments, as each environment represents different complexity trade-offs. The custom environment focuses on simplified physics for faster training in the multi-agent setting, where the DeepMind environment has more realistic physics (using the MuJoCo engine [32]). The RoboCup environment is the most realistic football engine of the three, as it includes properties such as stamina, fouls, kicking the ball out of bounds, communication and independent head movement. As mentioned, with random policies the custom environment is 25× faster than the RoboCup environment. When using the full 22 player training setup with neural network policies, the custom environment is still 14× faster than the RoboCup environment. It is also easier to learn in the custom environment as it has fewer time consuming dynamics such as throw-ins, fouls and players slowing down due to limited stamina. We can therefore iterate much faster on algorithmic ideas in the custom environment than what would otherwise be possible. We therefore focus on the custom environment for the rest of our experiments, and return to the RoboCup environment towards the end of this section.

5.2.2 Polyak averaged opponent

In Fig. 4, it can be seen that the 11 agent team first defeats the naive strategy team after 18 h of training, and then the teamwork strategy after 31 h. Ultimately, the learning team starts outperforming the best 2 versus 2 network (trained beforehand) after 45 h of training. Our team is completely trained using self-play in the full 11 versus 11 setting, with other strategies only being used for evaluations. This seems to indicate that training in the 22 player setting allows for better group strategy formation that would not usually be present in the 4 player setting.

5.2.3 League training

Here we train the system against a league of opponents. For the opponent trained in the 4 player setting we decided to use the best team derived when using Polyak averaging. We

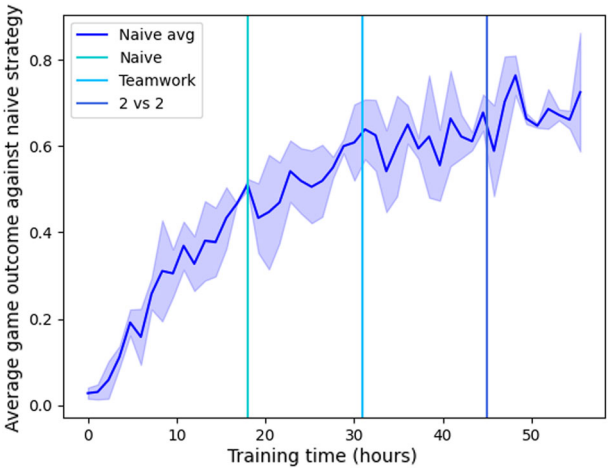


Fig. 4 The average game outcome (Naive avg) of a team of agents trained in the 11 versus 11 setting versus the naive strategy, over time (hours), with one standard deviation over three seed runs. A game outcome can be either a loss (0), draw (0.5) or win (1.0) for the team. The three vertical bars represent points where the team started outperforming a specific opponent team (achieving an average game outcome above 0.5). The 2 versus 2 opponent team (rightmost vertical bar) is the best network found after training in the 2 versus 2 player self-play environment also for 55 h. All evaluations are done in the 11 versus 11 setting

did this because the team training against a Polyak averaged opponent performed better than the one trained against a league after 55 h in the 4 player environment. In Fig. 5 we again plot the average game outcome of a team training in the 22 player setting, against the naive team, and plot vertical lines where the training team on average first outperforms a specific opponent. As can be seen, the team training against a league of opponents could still beat all the fixed opponents presented to it. We add the current training team's policy to the league of opponents every 10 iterations. When comparing the results in Fig. 5 to those in Fig. 4, it is apparent that the variance in training runs for the league opponent is higher than for the Polyak averaged opponent. Teams training against the Polyak averaged opponent also seem to learn slightly faster when comparing the win rates against the naive team.

In this section we trained our system in the full 11 versus 11 setting using Polyak averaged opponents and league opponents. Both setups managed to outperform the benchmark opponents in under 55 h of training and also outperformed the top team trained in the 2 versus 2 setting. This seems to indicate that it might be important to train in the 11 versus 11 setting for higher-level strategy formation. Furthermore, the Polyak averaged opponent setup seems to lead to less variance than the league opponent, and higher performance at almost every point throughout training.

A team training with Polyak averaged opponents performs well when evaluated against relatively weak higher-level strategies. However, it might not perform as well when higher-level strategies become more important. This is because the training team is learning to defeat only one average strategy. The league opponent strategy might be much less susceptible to this limitation, as it can learn to defeat many opponents with different strategies. Therefore, even though league training led to worse performance in the above experiments, it might give better results as the system improves and opponents become more challenging to beat.

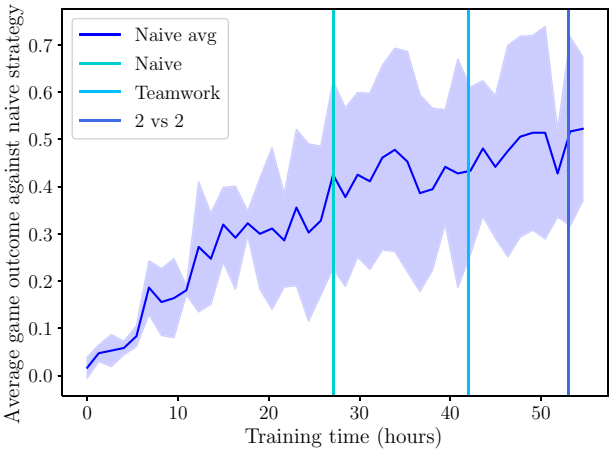


Fig. 5 The average game outcome of a team of agents trained in the 11 versus 11 setting against a league of opponents. All other settings are the same as was the case for Fig. 4

5.2.4 Ablation study

We now investigate the individual contributions of some of the main improvements on system performance. We do this to determine whether each component is necessary in the final system. We focus on training with a Polyak averaged opponent as it performed slightly better than the league opponent, and compare the complete system with two modified training systems. (1) We remove the Polyak averaged opponent and replace it with the latest policy. We also remove the freeze functionality for the opponent, such that the team of training agents is essentially playing against itself. (2) We replace the attention mechanism in the policy and critic networks with a feedforward layer, while keeping everything else the same. These two modified systems are evaluated because they are the main improvements that can influence the training profile (whether strategy collapse occurs or not). The other improvement methods are related to code acceleration where they execute mostly the same operations, but more efficiently. For the full system we take the training run with the median final score across three seeds.

In Fig. 6 we observe that the full system performs better than any of the modified systems. The team with feedforward networks instead of attention mechanisms, did not improve. The attention mechanism helps to significantly reduce the number of learnable parameters through added invariance assumptions which seems to allow for better strategy formation in this large multi-agent setting. The system without Polyak averaging did improve over time, but does so at a much slower pace. This seems to indicate that Polyak averaging and opponent freezing helps find more general strategies (evaluated against the naive handcrafted strategy which it never trains on) faster than pure self-play can. If the opponent is set to just be the current learning policy it becomes much easier for new behaviours, that perform worse, to be incorporated into the policy as both teams exhibit this new behaviour. Therefore the algorithm cannot tell from the reward signal that this strategy is worse than a previous one.

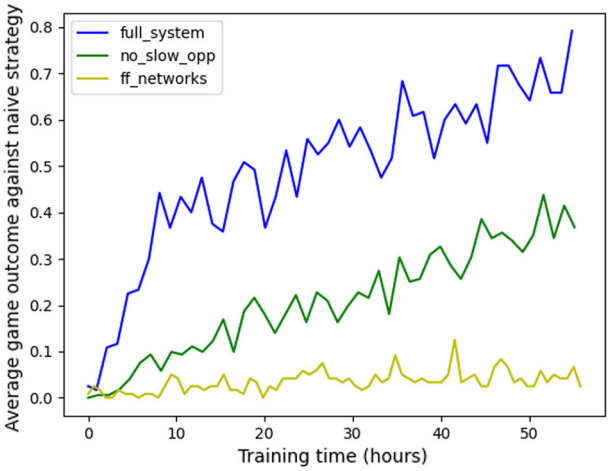


Fig. 6 Average score against the naive strategy over the course of training with different system configurations. These training runs are performed in the full 11 versus 11 setting. The three graphs represent the complete system (blue), the system without Polyak averaging or opponent freezing (green), and the system with feedforward networks instead of attention based networks (yellow). The full system achieves a higher score throughout training when compared to the modified systems (Color figure online)

Table 2 The average game outcomes between pairs of teams, after 500 games played in the full 11 versus 11 setting (Color table online)

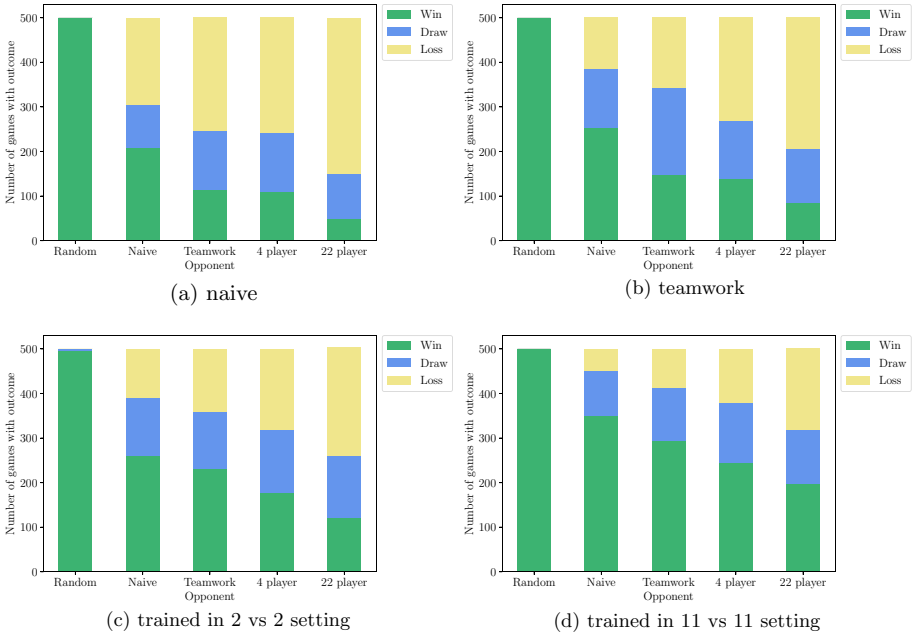
Team	Random	Naive	Teamwork	2 versus 2	11 versus 11
Random	0.50 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.04	0.00 ± 0.00
Naive	1.00 ± 0.00	0.51 ± 0.45	0.36 ± 0.41	0.35 ± 0.40	0.20 ± 0.33
Teamwork	1.00 ± 0.00	0.64 ± 0.41	0.49 ± 0.39	0.41 ± 0.42	0.29 ± 0.38
2 versus 2	1.00 ± 0.04	0.65 ± 0.40	0.59 ± 0.42	0.50 ± 0.42	0.38 ± 0.41
11 versus 11	1.00 ± 0.00	0.80 ± 0.33	0.71 ± 0.38	0.62 ± 0.41	0.52 ± 0.44

Each entry represents the average game outcome (0 for a loss, 0.5 for a draw, 1 for a win) that the row team achieves against the column team, plus-minus the standard deviation. We use the best performing seed for each team. The team trained in the 11 versus 11 setting outperforms all other teams

5.2.5 Training with different number of agents

Here we investigate how agents trained only in the 2 versus 2 player setting compare against agents trained in the full 11 versus 11 setting. We stop both algorithms after 55 h of training time have elapsed. Due to the policies being attention based, the agents trained in the 2 versus 2 setting can play in the 11 versus 11 setting. For the team trained in the 2 versus 2 setting, at the start of each game we spawn the agents at random starting positions. This allows them to better transfer to the full game as they have experienced each of the 11 field positions. The average scores of the different teams after training are given in Table 2.

As can be seen in Table 2, training in the full 11 versus 11 setting seems to yield the best team strategy when using the same computational resources. One obvious reason why agents trained in the 11 versus 11 setting might outperform agents trained in the 2 versus 2 setting, can be because an agent trained in the 2 versus 2 setting has never encountered more than 3 additional agents on the field. Therefore its attention policy might not be adapted to process



(a) naive (b) teamwork

(c) trained in 2 vs 2 setting (d) trained in 11 vs 11 setting

Fig. 7 The total games won, drawn and lost by each team against different opponents over 500 games

more agents effectively. To combat this we tried selecting the three agents that are closest to the ball, if the ball is visible, or otherwise randomly select up to three agents to pass to the policy. However, this made little difference to performance and still performs worse than the team of agents trained in the full 11 versus 11 setting.

We also present a breakdown of the total games won, drawn and lost by each team (except the random strategy) against different opponents. The results are given in Fig. 7.

As can be seen, the teamwork strategy has a high draw rate against itself. This seems to indicate that it has a strong defensive strategy, but lacks offensive capabilities, which makes sense as the strikers naively chase the ball. Along with having the highest average score, the team trained in the 11 versus 11 setting also has the highest game win rate against every opponents, and fewer draws. This might indicate that the team is more focused on offensive play than defensive play. We will investigate whether this is the case later in the section.

5.3 Team analysis

We next evaluate our trained teams to try and determine why the 11 versus 11 trained team outperforms the 2 versus 2 trained team.

5.3.1 Average field positions

Figures 8 and 9 show the average position of each trained agent over a number of games. Agents trained in the 11 versus 11 setting seem to remain in their positions longer, and spread out more, instead of simply charging towards the ball. This might also explain why agents trained in the 11 versus 11 setting perform better than those trained in the 2 versus 2 setting.

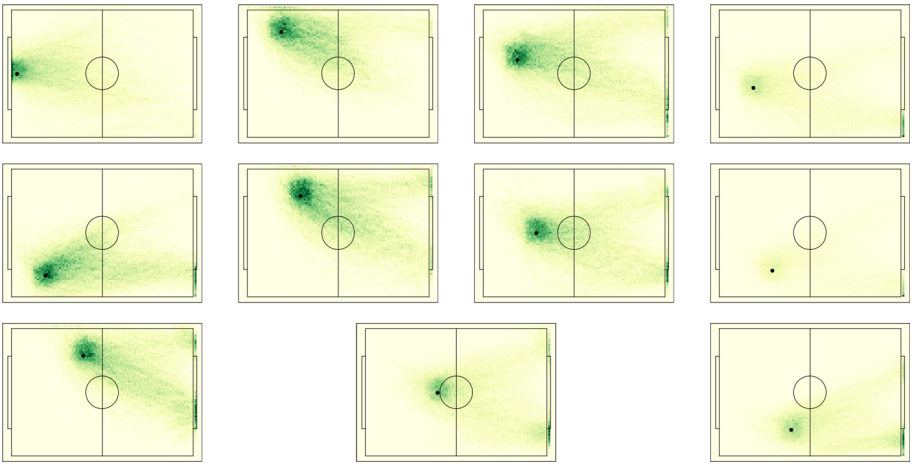


Fig. 8 Average player positions of agents trained in the 2 versus 2 setting through the course of an 11 versus 11 game. Agents spawn in specific positions depending on their shirt number, as indicated by the black circles on each image. The green–yellow colour intensities represent the probability of finding a specific player at that position throughout the game. Dark green represents more probable while light yellow is less probable. These agents seem to move from their initial positions quite quickly. The midfielders and attackers usually charge for the ball and try and score as quickly as possible, even if it means sacrificing defensive play (Color figure online)

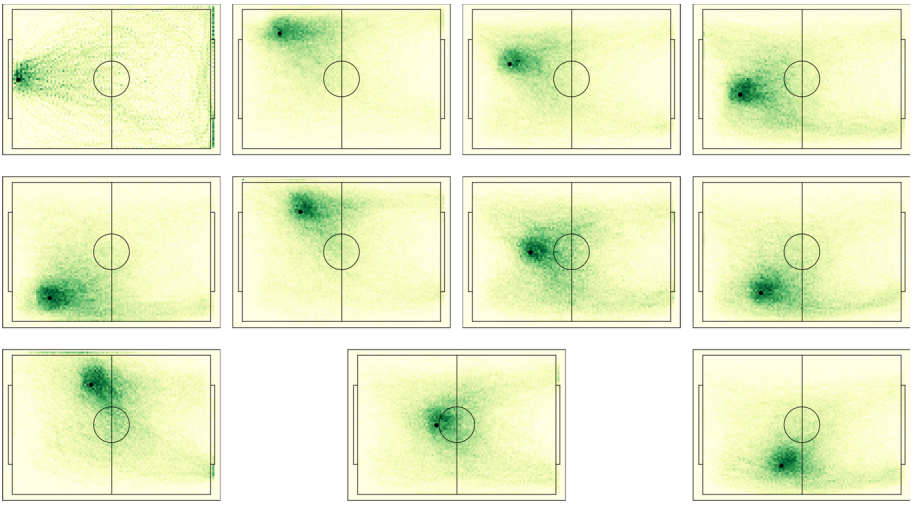
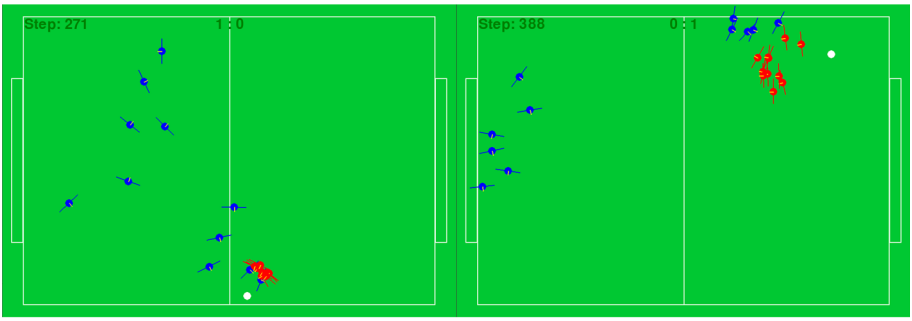
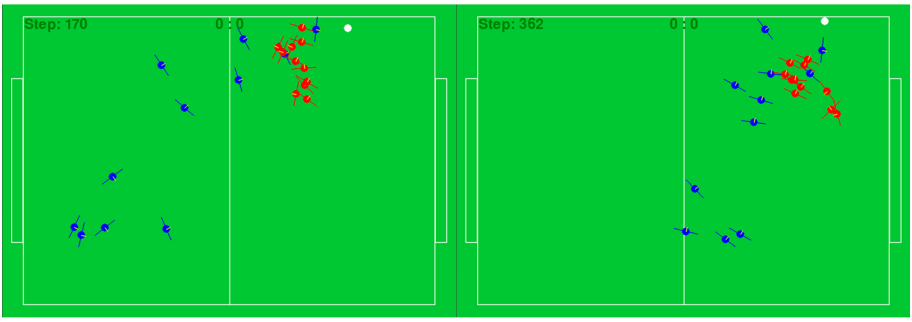


Fig. 9 Average player positions of agents trained in the 11 versus 11 setting. These agents seem to stay closer to their initial positions for a larger proportion of the game, indicating that they may have learned to play more positional football

In the 2 versus 2 case it becomes an all or nothing strategy. Every player moves towards the ball in an attempt to score a goal. However, if the ball passes them they might not be able to defend their own goal line. In the 11 versus 11 case they can still defend even if their strikers lose control of the ball.



(a) example 1 (b) example 2



(c) example 3 (d) example 4

Fig. 10 Defensive gameplay of the team trained in the 11 versus 11 setting (blue). The team is playing against the top team trained in the 2 versus 2 setting (red). Notice how the blue team splits into defensive and offensive groups (Color figure online)

5.3.2 Gameplay analysis

A video detailing various teams playing games against each other, with episode lengths of 400, is available online.[6] The video shows that the team trained in the 11 versus 11 setting exhibits less ball chasing behaviour than the team trained in the 2 versus 2 setting. The agents from the 11 versus 11 setting seem to either chase the ball if they are close to it, or position themselves in locations where the ball might be going to. As a result, this team tends to be much more spread out on the field than the team trained in the 2 versus 2 setting. This is seen as a good learned behaviour to have, as real football strategies usually involve remaining spread out over the field to allow for better defense, when the defending team does not control the ball, and better offence, as players have more options for where to pass the ball.

As demonstrated in Fig. 10, this learned positioning play sometimes leads to the team splitting into two groups on the field, akin to strikers and defenders.

While the team's positional play is not yet close to what a real football team is capable of, it is promising to see that this behaviour can be learned through self-play. It further seems to support the idea that training in the full 11 versus 11 setting is important for such strategies to emerge.

[6] Gameplay video: https://youtu.be/YrQVHeo9ZrA.

Table 3 Metrics of various teams playing against the team trained in the 11 versus 11 setting (Color table online)

Metric	Random	Naive	Teamwork	2 versus 2	11 versus 11
Move	0.50 ± 0.29	0.97 ± 0.02	0.55 ± 0.50	0.91 ± 0.13	0.54 ± 0.25
Rotation	0.20 ± 0.12	0.08 ± 0.11	0.06 ± 0.10	0.12 ± 0.10	0.08 ± 0.07
Ball distance	34.9 ± 18.2	16.6 ± 11.8	30.8 ± 22.36	16.4 ± 11.4	31.9 ± 19.3
Team distance	28.5 ± 12.5	8.39 ± 10.4	30.7 ± 20.8	8.62 ± 9.19	26.9 ± 14.6
Closest to ball	14.3 ± 11.9	9.20 ± 5.51	8.86 ± 4.78	9.05 ± 5.72	11.0 ± 10.1
Closest to goal	5.48 ± 2.72	35.6 ± 14.7	10.3 ± 1.70	31.3 ± 14.6	19.9 ± 12.8
Average outcome	0.00 ± 0.00	0.20 ± 0.33	0.29 ± 0.38	0.38 ± 0.41	0.52 ± 0.44

For the first two rows, a lower value is generally better as it implies that the players do not need to correct their positions that often. For rows 3 and 4, a higher value is generally better as it implies that the players are not bunching up. For rows 5 and 6, a lower value is generally better as it implies that at least one player is close to the ball or their own goal, respectively. For the average game outcome, higher values are better

5.3.3 Quantitative analysis

Next we consider a more quantitative approach to evaluate certain attributes of the various teams. We let each of the five teams (random, naive, teamwork, 2 vs. 2, and 11 vs. 11) play against the team trained in the 11 versus 11 setting and then track various metrics on their gameplay. We use the same opponent for all teams, as the metrics might change depending on who a team is playing against. The various metrics include the average move action output (in meters per step), the average rotational action output (in radians per step), the average distance of the team's players to the ball (in meters), the average distance of the team's players to each other (in meters), the average distance of the closest player to the ball (in meters), the average distance of the closest player to its own goal (in meters), and the average game outcome against the team trained in the 11 versus 11 setting (0 for a loss, 0.5 for a draw, and 1 for a win). These metrics are all measured over 500 games, and the results are presented in Table 3.

In combination, the metrics provide some indication of a team's positioning throughout gameplay. The handcrafted teamwork players, for example, have good positioning based on these metrics, yet their average game outcome (against the team trained in the 11 versus 11 setting) is relatively low. The average move metric is a bit deceiving for this team as the strikers do most of the ball chasing, and bunch up when doing so. The team trained in the 11 versus 11 setting on average has players relatively close to both the ball and their own goal. They are also generally more spread out on the field, and do not bunch up that often compared to the other teams, which might explain why they outperform all other teams.

5.4 Comparison to existing work

In this research, we focused on applying end-to-end MARL in the full game of football. We created a custom environment that executes more than $25\times$ faster than the 2D RoboCup simulator, even after removing all unnecessary wait instructions. MacAlpine and Stone [21] used MARL to allow their agents to learn individual football skills in the 3D RoboCup environment. However, their training was limited to only two agents. Liu et al. [18] also proposed a simplified environment with relatively fast simulation times. However, we cannot fairly compare our agents with theirs as they used 32 independent learning agents over

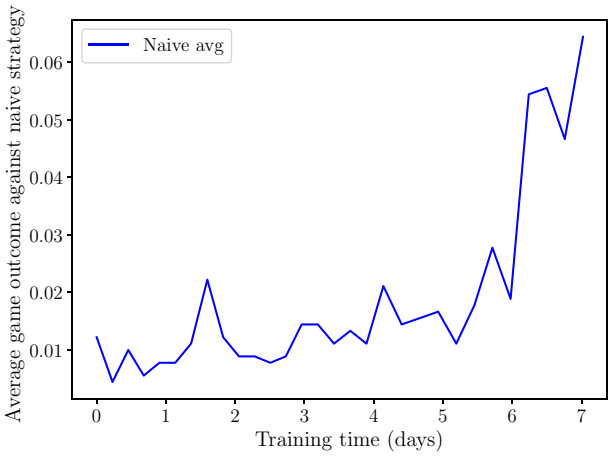


Fig. 11 Performance of a team training in the 2D RoboCup environment, in the 11 versus 11 setting, as evaluated against an opponent with the naive strategy. A game outcome can be either a loss (0), draw (0.5) or win (1) for the team

multiple GPUs for training. We limited ourselves to using only one GPU so that researchers with computational resource constraints can experiment with the implementations presented in this work. Furthermore, Liu et al. [18] only presented results with agents trained in the 2 versus 2 setting where we focused on stable training in the full 11 versus 11 setting. Lastly, we relied on a partially observable environment to more closely align with the 2D RoboCup setup, where Liu et al. [18] provided full state information to their agents.

5.5 Full system training in the 2D RoboCup environment

As a final experiment we consider whether it might be possible to execute our learning algorithm in the wrapped 2D RoboCup environment. This environment is 25× slower than our own custom environment, and 14× slower when neural network policies are used, which significantly impacts training times. For our algorithm to run in the RoboCup environment, we convert the hybrid action space to a simpler continuous action space (as described in Table 1). We use the same hyperparameters as those used for the Polyak averaged opponent experiment in Sect. 5.2. A RoboCup competition typically lasts around 6000 steps, which equates to 10 min of game time. For our training setup we train on only 400 steps per episode, to be consistent with our custom environment's training setup. After training the agents can play in games longer than 400 steps.

To enable learning in the brief initial reward shaping part of training, we remove the ball throw-ins, and instead allow the ball to bounce against the sides. This is needed because an opponent with a random policy would be unlikely to kick the ball back into play, and therefore stall the episode. The throw-ins are added back into the game after initial training when the reward shaping is removed.

For evaluation we implement a naive opponent team that has the same strategy as the naive team in the custom environment, i.e. chasing the ball and kicking it to the center of the opponent's goal. The result from training in the full 22 player RoboCup environment is presented in Fig. 11.

As can be seen, our algorithm takes significantly longer to train in the 2D RoboCup environment. However, after a few days of training the win rate against the naive opponent

does increase. Besides the environment step times, we also found that the actions in this RoboCup environment need to be more precise than in our custom environment. If the actions are too erratic, the team cannot consistently score goals. Therefore, for a large part of training the team does not score many goals, but after the noise (standard deviation) in action outputs lowers the agents start scoring more often.

Importantly, we did not observe any policy collapse through the course of training, which seems to indicate that the solutions proposed in this work also transfer to the RoboCup environment. However, because the environment is much slower than the custom environment, reaching competent policies takes much longer.

A video showing gameplay of these agents is also available online.[7] The naive team is still better than the team that is training, as the latter has not had enough time to learn to play a more positional game yet, and seems to also employ only a basic ball chasing strategy.

6 Discussion and future work

We investigated techniques needed to scale end-to-end MARL to the full 11 versus 11 player setting of simulated football. We believe this to be important for the development of more general learning algorithms, that both do well in competitions like RoboCup and can generalise to other domains. This aligns with the motivation behind RoboCup as a testing bed for algorithms that can one day be used to solve real-world problems. We further limited the computational resources that are required to train our agents so that other compute constrained researchers can build on this research. We also wanted to focus on generating football strategies entirely from self-play and not use other football agents to train against or perform supervised learning on. This self-play approach would be useful in domains that might not have good handcrafted or learned strategies available.

6.1 Summary of our approach and findings

By adapting the standard PPO algorithm with our proposed improvements we demonstrated that it is possible to train MARL agents directly in the full 22 player football game, in the self-play setting, using a limited computational budget. The trained team of MARL agents were evaluated against fixed strategies and achieved a higher average score than all other algorithms considered, after 46 h training on a single GPU. We performed an ablation study and showed that the opponent stabilisation mechanism and the use of attention networks help stabilise training in the 22 player setting. We showed the benefits of training in the 22 player setting over the 4 player setting. Using the same computational resources, the agents trained in the 22 player setting showed better performance than agents trained in the 4 player setting and evaluated in the full game. This seems to indicate that it is important to train in the full game. We showed that agents trained in the 22 player setting seemed to stay closer to their spawn positions than those trained in the 4 player setting, which might indicate better differentiation of individual responsibilities.

6.2 Our contributions

In this section we reflect on some of the contributions of this work.

[7] Gameplay video: https://youtu.be/OYL5fo6yiHY.

6.2.1 System design

Our first objective was to design a multi-agent system that can exhibit stable training in the full 22 player setting through self-play. To this end, we proposed various improvements to the PPO algorithm that would help it scale to the 22 player setting. We used an attention mechanism that can process a varying number of players, and scales linearly with the number of players, as opposed to the mechanism used by Liu et al. [19], which scales quadratically. The attention mechanism, along with the shared network setup, also reduces the number of parameters that need to be learned. To stabilise training we implemented two types of opponents, namely Polyak averaged opponents and a league of opponents. We showed through various experiments that these proposed improvements do enable stable training in the full 22 player setting.

6.2.2 Football environment

A common challenge in multi-agent settings is that training is extremely slow. Our second objective was to design a 2D football simulation environment, faster than RoboCup's, that retains the main challenges for multi-agent reinforcement learning. To achieve this objective, we created an environment where players could interact only with the ball and not also each other. Furthermore, the ball bounces from the side walls so that no throw-ins are needed. The environment uses matrix multiplication to calculate all player position updates and observation information in parallel. This drastically improves the throughput of the environment. The end result is an environment that is 25$\times$ faster than the 2D RoboCup environment.

6.2.3 Evaluation

Our third objective was to experimentally verify that our solution can learn competent strategies when evaluated against handcrafted opponents. We could see that our system using both the Polyak averaging and league training setups could learn without policy collapse. We found that our team, trained in the 22 player setting, outperformed all handcrafted opponents and also a team trained in the 4 player setting. This confirmed our suspicion that training in the 22 player setting can allow for higher-level strategies to form. We further evaluated the behaviour of the team trained in the 22 player setting and found that it exhibits more positional play than the team trained in the 4 player setting. The 22 player trained team tends to spread out more and sometimes forms two groups, one defensive and one offensive. While their strategies are not yet close to those of human teams, it is promising to see these positional play strategies emerge from self-play alone.

6.3 Future work

It seems our trained agents can still be improved upon, when visually comparing their gameplay to the top teams in 2D RoboCup competitions. These top teams have much better positional play with more directed ball passing. While leaving our learning team to train for longer does yield better strategies, the rate of increase in the team's performance diminishes. To speed up improvement it might be worth training with a varied number of players on the field. This would allow for low-level ball handling skills to be learned in the leagues with fewer players. The full 22 player environment can then still be used to learn high-level team strategies.

Another possible explanation for what might be limiting our current MARL approach could be that the agents need to better assign individual responsibilities inside the context of the team, which might also require a communication mechanism to dynamically share information. Furthermore, it might be a good idea to use a recurrent critic to identify an opponent or gauge its strength. This could allow for faster learning as expected rewards can be estimated with greater accuracy.

We would also like to investigate adding communication to our team, which should help them to coordinate better. Lastly, we would like to improve our end-to-end learning system to learn faster in the 2D RoboCup simulator, with the ultimate goal of creating a competent team that existing participants find challenging to beat, while still relying only on MARL through self-play. This can potentially inspire others in applying end-to-end MARL to other domains.

Funding Open access funding provided by Stellenbosch University.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Appendix A Hyperparameter settings

See Tables 4 and 5.

Table 4 Hyperparameters used in the 2 versus 2 and 11 versus 11 training settings

Parameter	Value
Critic loss weighting (α_v)	0.5
Policy loss weighting (α_p)	1.0
Entropy loss weighting (α_e)	0.01
Polyak averaging update rate (α_s)	0.01
PPO clipping value (ϵ_p)	0.1
Opponent update threshold (ϵ_s)	0.55
Reward discount factor (γ)	0.999
epochs_per_step	5
Learning rate in the Adam optimiser [13]	10^{-4}
Experience batch size	660
Mini-batch size	60
Episode length	400
LSTM layer size	32
Feedforward layer size	64

Table 5 Neural network layers used by the policy and critic

Name	Type	Description
`encode_ff`	2-layer feedforward neural network	Used to encode general information such as the ball's location and time information
`query_ff`	2-layer feedforward neural network	Used to generate query embeddings for the multi-headed attention layer
`encode_team_ff`	2-layer feedforward neural network	Used to generate team embeddings
`encode_opp_ff`	2-layer feedforward neural network	Used to generate opponent embeddings
`value_opp_ff`	2-layer feedforward neural network	Used to generate values for the multi-headed attention layer
`out_ff`	2-layer feedforward neural network	Produces a final scalar output representing the critic's estimate for the cumulative reward remaining in the episode
`out_r`	2-layer LSTM network	Produces weightings for a probability distribution over the actions an agent can take

References

1. Akiyama, H., Nakashima, T., Tanaka, S., & Fukushima, T. (2017). HELIOS2017: Team description paper. *RoboCup 2017: Robot World Cup XXI*.
2. Akiyama, H., Nakashima, T., Fukushima, T., Suzuki, Y., & Ohori, A. (2019). HELIOS2019: Team description paper. *RoboCup 2019: Robot World Cup XXIII*, 2019.
3. Bansal, T., Pachocki, J. W., Sidor, S., Sutskever, I., & Mordatch, I. (2018). Emergent complexity via multi-agent competition. *ArXiv Preprint*, arXiv:1710.03748.
4. Berger, U. (2007). Brown's original fictitious play. *Journal of Economic Theory, 135*, 572–578.
5. Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., Pinto, H. P. D. O., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., & Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning. *ArXiv Preprint*, arXiv:1912.06680, 2019.
6. Brown, G. (1951). Some notes on computation of games solutions. *RAND Corporation Report*.
7. Busoniu, L., Babuska, R., & De Schutter, B. (2008). A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics Part C (Applications and Reviews), 38*, 156–172.
8. Devlin, S., & Kudenko, D. (2016) Plan-based reward shaping for multi-agent reinforcement learning. In *IEEE conference on intelligent systems* (pp. 44–58).
9. Devlin, S., Yliniemi, L., Kudenko, D., & Tumer, K. (2014). Potential-based difference rewards for multiagent reinforcement learning. In *Proceedings of the thirteenth international joint conference on autonomous agents and multiagent systems*.
10. Foerster, J. N., Farquhar, G., Afouras, T., Nardelli, N., & Whiteson, S. (2018). Counterfactual multi-agent policy gradients. In *Proceedings of the thirty-second AAAI conference on artificial intelligence and thirtieth innovative applications of artificial intelligence conference and eighth AAAI symposium on educational advances in artificial intelligence*.
11. Heinrich, J., Lanctot, M., & Silver, D. (2015). Fictitious self-play in extensive-form games. In *International conference on machine learning*.

12. Hoffman, M., Shahriari, B., Aslanides, J., Barth-Maron, G., Behbahani, F., Norman, T., Abdolmaleki, A., Cassirer, A., Yang, F., Baumli, K., Henderson, S., Novikov, A., Colmenarejo, S. G., Cabi, S., Gülçehre, Ç., Le Paine, T., Cowie, A., Wang, Z., Piot, B., & De Freitas, N. (2020). Acme: A research framework for distributed reinforcement learning. *ArXiv Preprint*, arXiv:2006.00979.
13. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *ArXiv Preprint*, arXiv:1412.6980.
14. Kirkpatrick, J., Pascanu, R., Rabinowitz, N. C., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2016). Overcoming catastrophic forgetting in neural networks. *ArXiv Preprint*, arXiv:1612.00796.
15. Lanctot, M., Zambaldi, V. F., Gruslys, A., Lazaridou, A., Tuyls, K., Pérolat, J., Silver, D., & Graepel, T. (2017). A unified game-theoretic approach to multiagent reinforcement learning. *ArXiv Preprint*, arXiv:1711.00832.
16. Lawson, C. L. (1972). Transforming triangulations. *Discrete Mathematics, 3*, 365–372.
17. Leslie, D. S., & Collins, E. J. (2006). Generalised weakened fictitious play. *Games and Economic Behavior, 56*, 285–298.
18. Liu, S., Lever, G., Merel, J., Tunyasuvunakool, S., Heess, N., & Graepel, T. (2019). Emergent coordination through competition. In *International conference on learning representations*, 2019. https://doi.org/10.48550/ARXIV.1710.03748.
19. Liu, S., Lever, G., Wang, Z., Merel, J., Eslami, S. M. A., Hennes, D., Czarnecki, W. M., Tassa, Y., Omidshafiei, S., Abdolmaleki, A., Siegel, N. Y., Hasenclever, L., Marris, L., Tunyasuvunakool, S., Song, H. F., Wulfmeier, M., Muller, P., Haarnoja, T., Tracey, B. D., Tuyls, K., Graepel, T., & Heess, N. (2021). From motor control to team play in simulated humanoid football. *ArXiv Preprint*, arXiv:2105.12196.
20. Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. *ArXiv Preprint*, arXiv:1706.02275.
21. MacAlpine, P., & Stone, P. (2018). Overlapping layered learning. *Artificial Intelligence, 254*, 21–43.
22. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature, 575*, 350–354.
23. Omidshafiei, S., Pazis, J., Amato, C., How, J. P., & Vian, J. (2017). Deep decentralized multi-task multi-agent reinforcement learning under partial observability. *ArXiv Preprint*, arXiv:1703.06182.
24. OpenAI. (2019). Dota 2 with large scale deep reinforcement learning. *ArXiv Preprint*, arXiv:1912.06680.
25. Pan, H.-R., Gürtler, N., Neitz, A., & Schölkopf, B. (2021). Direct advantage estimation. *ArXiv Preprint*, arXiv:2109.06093.
26. Pretorius, A., Tessera, K., Smit, A. P., Formanek, C., Grimbly, S. J., Eloff, K., Danisa, S., Francis, L., Shock, J., Kamper, H., Brink, W., Engelbrecht, H., Laterre, A., & Beguir, K. (2021). Mava: A research framework for distributed multi-agent reinforcement learning. *ArXiv Preprint*, arXiv:2107.01460.
27. Prokopenko, M., & Wang, P. (2019). Guiding self-organisation of intelligent agents (Fractals2019). *RoboCup 2019: Robot World Cup XXIII*.
28. RoboCup. (2021) RoboCupSoccer. https://www.robocup.org/domains/1. [Online; accessed 22-August-2022].
29. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *ArXiv Preprint,* arXiv:1707.06347.
30. She, J., Gupta, J. K., & Kochenderfer, M. J. (2022). Agent-time attention for sparse rewards multi-agent reinforcement learning. *ArXiv Preprint*, arXiv:2210.17540.
31. Stone, P., & Veloso, M. (2000). Layered learning. In *European conference on machine learning* (pp. 369–381).
32. Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *International conference on intelligent robots and systems*.
33. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems*.
34. Wu, Z., Yu, C., Ye, D., Zhang, J., Piao, H., & Zhuo, H. H. (2021). Coordinated proximal policy optimization. *ArXiv Preprint*, arXiv:2111.04051.
35. Yang, F., Barth-Maron, G., Stanczyk, P., Hoffman, M. W., Liu, S., Kroiss, M., Pope, A., & Rrustemi, A. (2021) Launchpad: A programming model for distributed machine learning research. *ArXiv Preprint*, arXiv:2106.04516.
36. Yang, Z., Preuss, M., & Plaat, A. (2021). Potential-based reward shaping in sokoban. *ArXiv Preprint*, arXiv:2109.05022.
37. Yu, C., Velu, A., Vinitsky, E., Yu, W., Bayen, A. M, & Wu, Y. (2021). The surprising effectiveness of MAPPO in cooperative, multi-agent games. *ArXiv Preprint*, arXiv:2103.01955.
38. Zare, N., Sayareh, A., Sarvmaili, M., Amini, O., & Soares, A. (2021). CYRUS soccer simulation 2D team description paper 2021. *RoboCup 2021: Robot World Cup XXV*.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.